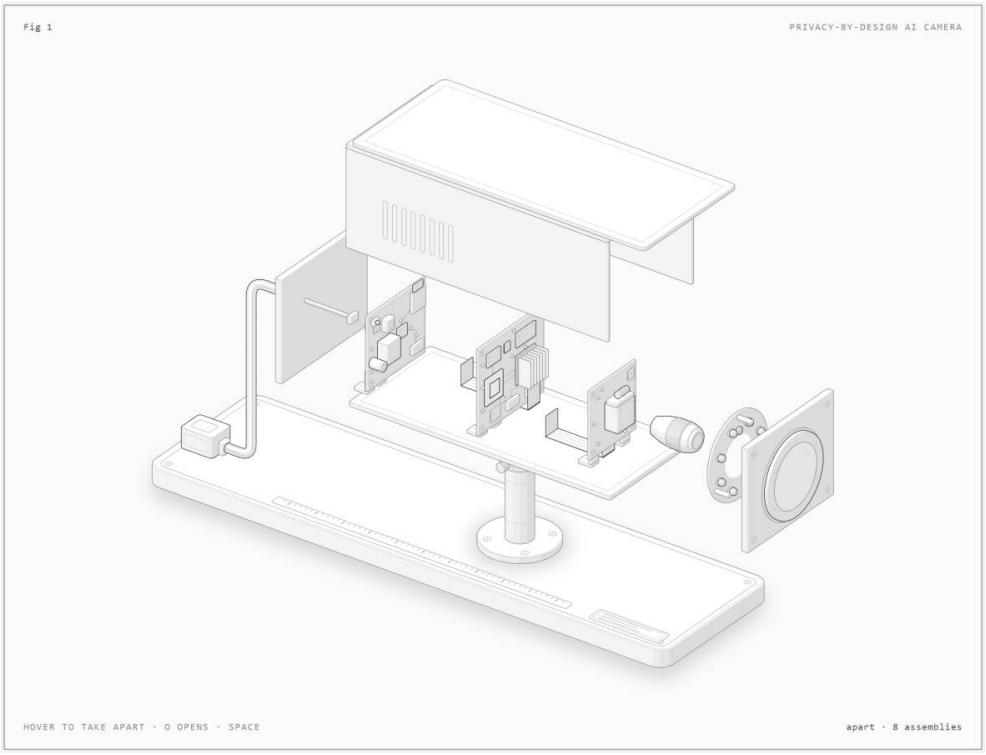
MINI PROJECT / CASE STUDY REPORT

Privacy-by-Design AI CCTV

On-camera masking of intimate body regions and UPI PIN entry using pose estimation, body-part detection and hand tracking



AI application area: AI-based image recognition (edge computer vision)

Submitted by: Leon Kaushik Deka

Third Semester Mini Project

Artificial Intelligence and Machine Learning (AIML)

October 2026

Contents

1. Introduction
2. Problem Statement
3. AI Approach
4. Data
5. Techniques and Algorithms
6. Working Process
7. Implementation and Demonstration
8. Results and Applications
9. Advantages and Limitations
10. Ethical Considerations
11. Future Scope
12. Conclusion

References

Appendix A: Jupyter notebook screenshots

Appendix B: Data sources and licences

Abstract

There are CCTV cameras everywhere: hostels, banks, shops. The issue is that these cameras often record too much. Take a CCTV camera in a hostel corridor. It records residents in personal clothing. Or a camera above a billing counter. It records customers entering their UPI PIN. And who can access this video? Anyone with access to the control room, or an attacker who can hack into the system.

This is a project to create an AI/ML CCTV pipeline that only masks the sensitive areas in the video. All the frames go through the AI chip in the camera. This means that there is no way to store or transmit video without the masking. We use pose estimation along with a pre-trained body part detector to locate and mask nipples, buttocks and genitalia. We have a second pipeline that detects phones and POS machines and with the help of hand tracking, we can detect when a PIN is being entered and mask both the screen and the hand. Masks are permanent. Since there is no forensic value in a PIN or a private body part, the rest of the video is still good as evidence.

A laptop-and-webcam prototype was built and tested on 55 hand-labelled, openly licensed images. With the default policy it fully protected 4/8 PIN-entry regions, raised 1/26 false alarms on images of idle devices and 0/21 on ordinary clothed people, and processed 640x480 frames of a busy scene at 7.2 fps on a laptop CPU with no GPU or NPU. Every miss traced back to a hand or a device that the pre-trained detectors did not find, which sets out clear next steps.

1. Introduction

Video surveillance is one of the most common camera uses we come across in daily life. Hostels, banks, ATMs, retail billing counters, hospitals and offices all record continuously, and the recordings are kept for weeks. Today's IP cameras are mini computers: they have a system-on-chip (SoC) with an image signal processor (ISP), a video encoder and, more and more often, a neural processing unit (NPU) designed to run deep-learning models on the camera itself (edge AI).

Usually on-camera AI is used to add surveillance features like people counting or intrusion alerts. This project turns the same hardware around and uses it to remove data the camera should never have captured. It's an application of AI-based image recognition: four pre-trained computer vision models (human pose estimation, body-part detection, object detection and hand landmark tracking) combined with rule-based fusion, temporal reasoning and irreversible image masking.

The main principle is privacy by design [15]. Privacy is baked into the design, not added as a policy after the fact. And if you mask before encoding, inside the camera, no unmasked recording exists anywhere. Nothing to leak, hand over under subpoena, misuse or steal.

2. Problem Statement

CCTV cameras record more than they need to, and the extra is the most sensitive part:

- **Hostels and dormitories.** Corridor cams. They see residents moving between rooms and bathrooms in towels or underwear. The camera is there for safety, not to watch bodies.
- **Billing counters, banks and ATMs.** A camera installed above the counter can capture the exact finger positions of a customer typing their UPI PIN on a phone, or their card PIN on a POS terminal. Once someone has the PIN, a stolen phone or card can be drained.

- **Access is wide.** It can be accessed by maintenance vendors, control room personnel and anybody else who gets an export. Also IP cameras are often a target of attack. The Mirai botnet exploited many IP cameras with default logins [12].

If you blur entire frames or turn the cameras off, you lose the security benefit the cameras exist for. So the challenge is to remove only the sensitive parts, in real time, in a way that cannot be reversed, while keeping the rest of the frame as evidence. Three measurable questions are used to judge the prototype:

1. How well does it mask the sensitive areas (protection)?
2. How often does it mask areas that did not need masking (over-masking / false alarms)?
3. How quickly can it process frames in real time (latency / frames per second)?

3. AI Approach

It's a per-frame vision pipeline meant to run on the camera's NPU, between the image sensor and the video encoder. There are two detection branches and a single masking stage shared by both.

- **Body privacy.** A pose estimation model finds the skeleton of each person. The skeleton tells us where a chest and a pelvis can be, and it guides a pre-trained body-part detector. The detector runs on each person at full resolution. Detections inside the matching anatomical region are accepted more easily, and anatomically impossible ones are discarded. Exposed nipples, buttocks and genitalia are pixelated.
- **PIN guard.** An object detector finds phones and POS terminals. A hand tracker gives 21 keypoints per hand. When a fingertip lands on the face of a device, the device and the hand are filled solid until the interaction ends.

Four design decisions turn detection into privacy protection:

- **Mask at the source.** Masks are written into the frame buffer before encoding, so the only video that exists is the masked video.
- **Permanent masks.** Solid fill or coarse pixelation, never Gaussian blur (which can be partly reversed).
- **Mask early, release late.** A region is masked as soon as there is evidence, and released only after 12 consecutive frames without evidence. This way detector flicker can't reveal individual frames.
- **Fail closed.** If any model fails on a frame, the whole frame is withheld instead of passed through.

Why do you need AI? Because the sensitive areas can move. And change shape. And show up anywhere in the frame. A static privacy zone (those black rectangles you set up in your camera settings) is not going to follow you as you walk down a hallway. And isn't going to follow the phone in your customer's hand. And it's obscuring way more of the picture than you need.

4. Data

4.1 Data the system consumes

- **Input:** colour video frames from the camera sensor (the prototype uses 640x480 webcam frames).
- **Intermediate:** 33 keypoints for each person. 21 keypoints for each hand. Bounding boxes (with class, confidence score) for body parts, devices.
- **Output:** the same frames with the sensitive regions permanently masked, plus metadata only (number of masked regions, timings). No unmasked pixels are stored.

4.2 Data the models were trained on

All four models were pre-trained by their authors; no training was done in this project. EfficientDet-Lite0 was trained on COCO 2017 [5], which has 80 classes, including 'cell phone' and 'remote'. The MediaPipe pose and hand models were trained by Google on large annotated datasets of body and hand images [2, 3]. The NudeNet detector was trained by its author on a body-part dataset with 18 classes (exposed and covered) [7].

4.3 Test data used in this project

We did not attempt to gather any video of real people taking off their clothes or real PINs. We could not have done so without infringing on the privacy we want to protect. 83 openly licensed photos were downloaded from Wikimedia Commons (list of licences per image in Appendix B). 55 were manually labelled to form three sets:

Set	Images	Meaning	Label
A	8	a hand operating a phone or POS terminal	box over device face + operating fingers
B	26	phone / POS / PIN pad visible, nobody entering anything	must not be masked
C	21	ordinary clothed people in corridors and shops	must not be masked

Ambiguous regions, such as a customer scanning a QR code, are labelled 'ignore': masking them counts neither as protection nor as over-masking. A production system would also need consented, annotated CCTV recordings of real counters and corridors, plus a labelled POS-terminal dataset. The webcam app in this project can record such data with frame-level ground truth (see Section 7).

5. Techniques and Algorithms

5.1 Pre-trained models

Task	Model	Architecture	Input	Output	Size
Pose estimation	MediaPipe Pose Landmarker (lite)	BlazePose: person detector + landmark regression CNN [2]	256x256	33 keypoints / person	5.8 MB
Body parts	NudeNet v3 '320n' [7]	YOLOv8n single-stage detector, ONNX	320x320	18 classes (exposed / covered)	12.2 MB
Phones / POS	EfficientDet-Lite0, int8 [4]	EfficientNet-Lite backbone + BiFPN	320x320	COCO boxes: cell phone, remote	4.6 MB
Hand tracking	MediaPipe Hand Landmarker [3]	BlazePalm detector + landmark CNN	192/224 px	21 keypoints / hand	7.8 MB

5.2 Algorithms developed in this project

- **Pose-guided ROI cropping.** Each person's chest-to-thigh box is cropped and passed to the body-part detector, so a person at the end of a long corridor is seen at the detector's full 320 px instead of a few pixels.
- **Region-aware thresholding.** The pipeline uses shoulder and hip keypoints to work out chest and pelvis areas. A breast detection inside a chest area, or a genitalia/buttocks detection inside a pelvis area, is accepted at a score of 0.25. Anywhere else it needs 0.45.
- **Vetoes against detector errors.** A body-part detection that lies mostly on a tracked hand is rejected unless it sits inside a person's matching body region (the detector mistakes fingers for genitalia). On an upright person, a pelvis part above the shoulders or a chest part above the nose is anatomically impossible and is rejected. Skeletons with implausible proportions aren't trusted (the pose model sometimes builds a 'person' out of two hands).
- **Pose-anchored persistence.** Each accepted body part is stored relative to the person's hip centre and torso length. So if the detector misses a frame, the mask moves with the body.
- **PIN-entry rule.** A hand is entering a PIN when an index, middle or thumb fingertip touches the device face (device box + 12 %). An extended pointing index finger, a thumb on a held phone, or fingertip tapping over the last 10 frames raises the confidence. The rule is deliberately high-recall.
- **Guided zoom passes.** Small hands and phones get a second look at full model resolution: device -> crop for the hand, pose wrist -> crop for the hand, hand -> crop for the device. Only targets smaller than 20 % of the frame height are zoomed, with at most 3 crops per pass on still images and 1 per frame on live video (cycling through candidates; the temporal hold covers the gap).
- **Parallel branches.** Pose -> body parts, the device detector and the hand tracker don't depend on each other within a frame, so they run in parallel, like separate engines on a camera SoC.
- **Call filter.** A device sitting on a head (a pose head box or a confidently detected face) is a phone call, not PIN entry. Grazing the chin is not enough: the device centre must be on the head, or most of the device must overlap it.
- **Occlusion memory.** The last device box is stored relative to the wrist. While the hand hides the phone, the device mask moves with the hand.
- **Temporal hold.** Mask regions are tracked by IoU matching and kept for 12 frames after the last detection.
- **Irreversible masking and fail-closed.** PIN regions are filled solid, because even a pixelated finger can show which row of keys is being pressed. Body regions are pixelated to about 6 cells across. Any exception blanks the whole frame.

5.3 Per-frame algorithm

```
for each frame F from the sensor:
    try:
        people  = pose(F);         parts   = body_parts(F) + body_parts(crop(F, roi(p)) for p in people)
        devices = detect(F);       hands   = hand_tracker(F)
        hands  += zoom_hands(F, devices, wrists(people));  devices += zoom_devices(F, hands)
        body = gate(parts, regions(people), veto_hands=hands) + anchored_parts(people)
        pin  = [hand U device for hand, device if fingertip_on(device, hand)
                and not at_head(device)] + carried_devices(hands)
        masks = hold(body) + hold(pin)          # 12-frame release delay
        F[masks] = solid / pixelate              # in place, before encoding
```

```
except error:
    F[:] = 0                                # fail closed
encoder.write(F)
```

6. Working Process

The figure below follows one frame from input to output. Both branches work on every frame. The PIN guard gets pose keypoints (wrist crops, head boxes), and the body branch gets the tracked hands (hand veto).

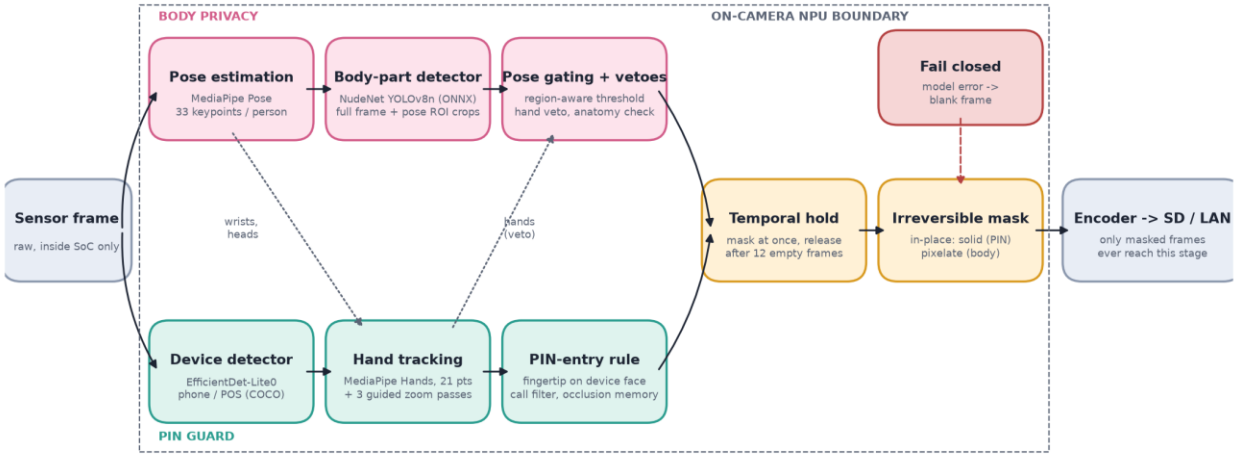


Figure 1. Per-frame workflow inside the camera's NPU boundary.

6.1 Inside the camera

In the exploded view below you can see where the pipeline sits in an actual AI camera: light comes through the front window and lens, the IR-cut filter switches for day and night, and the CMOS image sensor sends raw frames through a flexible cable to the main SoC. The ISP turns them into colour frames, the NPU runs the privacy pipeline and writes masks into the frame buffer, and only after that does the H.265 encoder read the frame for the microSD card and the PoE network port. There is no physical path from the sensor to storage or the network that bypasses the NPU stage.

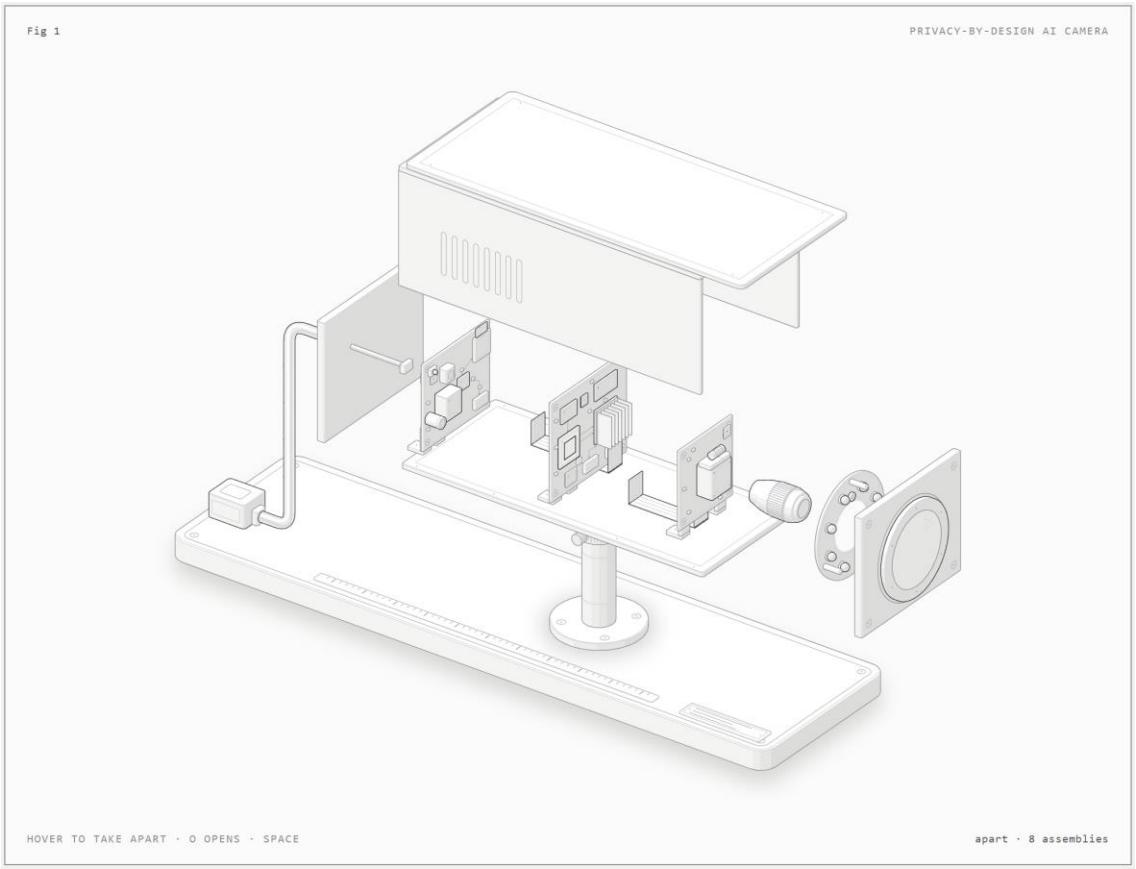


Figure 2. Exploded view of the privacy-by-design AI camera: front window, IR LED ring, M12 lens, image sensor board, main board (SoC + NPU), I/O board (PoE, microSD), rear cap and cover. Interactive version: anatomy/cctv-camera/cctv-camera.html.

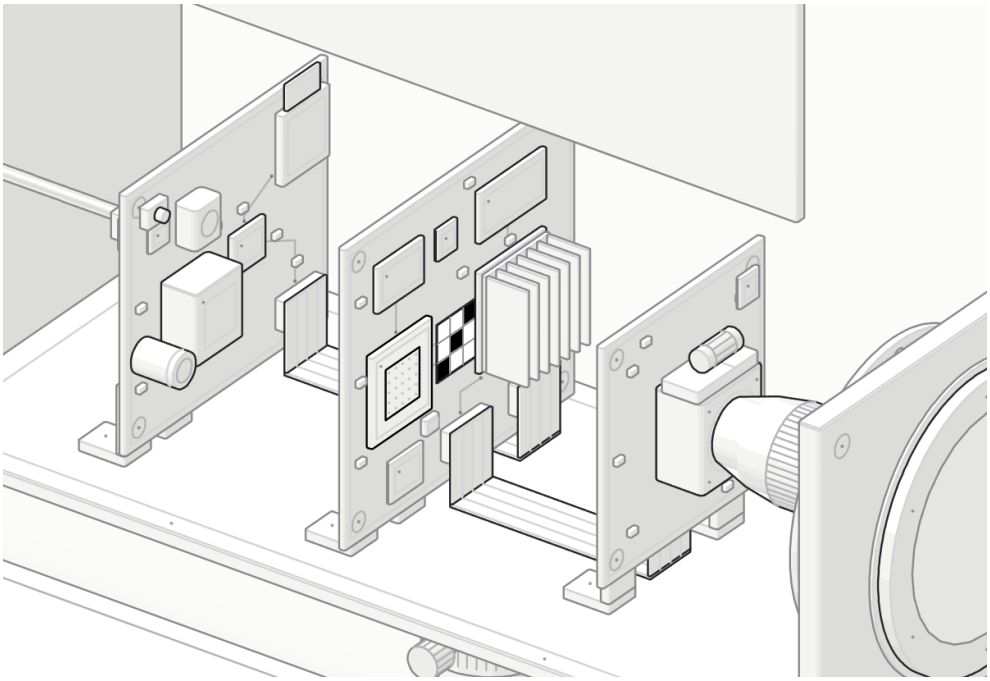


Figure 3. Close-up of the boards. The frame tile has left the sensor board on the flat flex cable and sits on the NPU with its sensitive regions already blacked out; only this masked frame continues to the encoder and the I/O board.

6.2 Step by step

4. Capture: the sensor delivers a frame into a buffer that only the pipeline can read.
5. Pose: up to 4 skeletons are estimated; chest, pelvis, head and wrist positions are derived.
6. Body parts: the detector runs on the whole frame and on each person crop; detections pass the gates and vetoes.
7. Devices and hands: phones/POS terminals and hands are detected, then re-checked with zoom crops.
8. PIN rule: fingertip-on-device interactions become PIN masks (calls excluded, hidden devices carried).
9. Hold: new regions mask immediately; old regions persist 12 frames after their last evidence.
10. Mask: regions are filled or pixelated in place. On any error the frame is blanked instead.
11. Encode: the masked frame is compressed and written to storage or streamed. The raw buffer is overwritten by the next frame.

7. Implementation and Demonstration

Python 3.12 with OpenCV, MediaPipe Tasks and ONNX Runtime. Runs on a laptop CPU, with a webcam standing in for the camera sensor. All models run locally. No cloud.

Path	Contents
privacy_cctv/detectors/	wrappers for the four pre-trained models (vectorised NudeNet decoder)
privacy_cctv/body_privacy.py	pose-guided cropping, gating, vetoes, anchored persistence
privacy_cctv/pin_guard.py	PIN-entry rule, call filter, occlusion memory, tapping cue
privacy_cctv/memory.py, masking.py	temporal hold; irreversible masking
privacy_cctv/pipeline.py	per-frame orchestration, zoom passes, fail-closed, stage timings
privacy_cctv/edge_camera.py	simulated camera boundary: only masked frames leave
privacy_cctv/metrics.py	protection, over-masking, false alarms, leak rate, latency
app/run_camera.py	live webcam prototype with recording and ground-truth logging
notebooks/privacy_cctv_demo.ipynb	stage-by-stage demonstration and evaluation
tests/	26 unit tests (geometry, masking, hold, PIN rule, call filter, fail-closed)
anatomy/cctv-camera/	interactive exploded view of the camera

7.1 How to run

```
python -m venv .venv  &&  .venv\Scripts\pip install -r requirements.txt
python scripts/fetch_assets.py --samples        # models + sample images
jupyter lab notebooks/privacy_cctv_demo.ipynb   # demonstration and evaluation
python app/run_camera.py --source 0 --profile retail --log data/runs/demo.csv
python -m pytest tests                          # 26 tests
```

In the live app, press g while you are actually typing a PIN to mark ground truth. The per-frame CSV log then gives frame-level protection, false masking and FPS for your own recording. The last notebook cell analyses it.

7.2 Demonstration (Jupyter notebook)

The notebook executes each step on real images. Here are some chosen results; all cells can be found in Appendix A.

```
In [3]: pipe = PrivacyPipeline(PrivacyConfig())
        img = load("phone_06.jpg")
        r = pipe.process(img.copy())
        vis = overlay(img, r)
        show(label_strip(vis, "Pose keypoints -> chest and pelvis regions"), width=6, save="stage_pose.png")
        p = r.people[0]
        pd.DataFrame([{"region": k, "x1": round(b.x1), "y1": round(b.y1), "x2": round(b.x2), "y2": round(b.y2)}
                      for k, b in p.regions().items()]).assign(torso_px=round(p.torso_len))
```



Out[3]:

	region	x1	y1	x2	y2	torso_px
0	chest	160	213	305	334	221
1	pelvis	139	377	275	553	221

Figure 4. Notebook: pose keypoints and the chest and pelvis regions derived from them.

```
In [5]: demo_cfg = PrivacyConfig(pin_privacy=False,
                                 sensitive_parts=("FEMALE_BREAST_COVERED", "BUTTOCKS_COVERED", "FEMALE_GENITALIA_COVERED"))
        demo = PrivacyPipeline(demo_cfg)
        tiles = []
        for name in ["phone_06.jpg", "phone_01.jpg"]:
            im = load(name); demo.reset()
            rr = demo.process(im.copy())
            tiles += [label_strip(overlay(im, rr), f"{name}: model view"), label_strip(rr.frame, "camera output")]
        show(side_by_side(*tiles, height=420), "Demonstration: pose-gated masking (covered classes used as stand-ins)",
             save="stage_body_mask_demo.png")
```

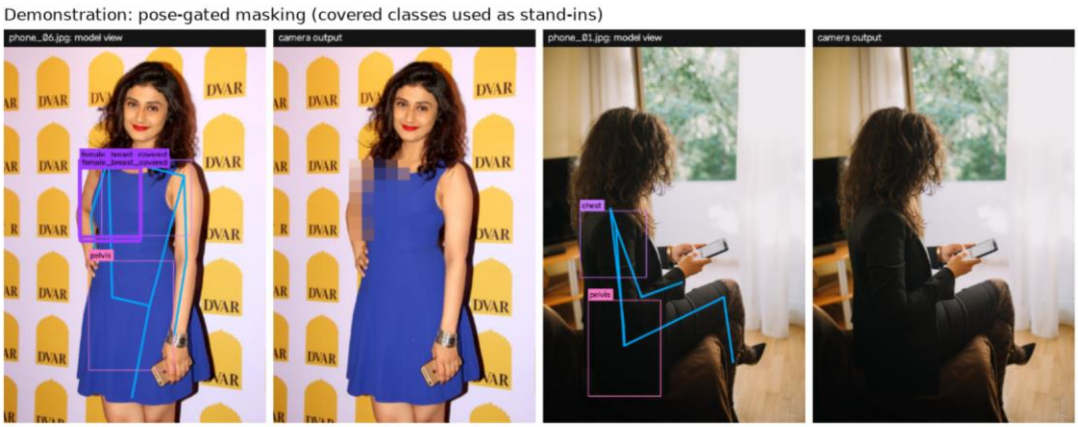
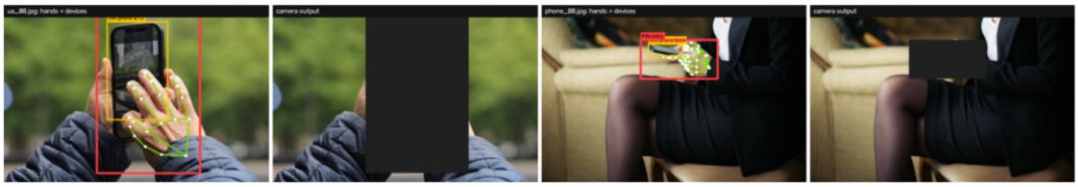


Figure 5. Notebook: pose-gated body masking, demonstrated with the covered classes used as stand-ins (no explicit imagery was used anywhere in this project).

In [6]:
```
pin_pipe = PrivacyPipeline(PrivacyConfig(body_privacy=False))
tiles, rows = [], []
for name in ["ua_00.jpg", "phone_00.jpg", "ue_00.jpg"]:
    im = load(name); pin_pipe.reset()
    rr = pin_pipe.process(im.copy())
    tiles += [label_strip(overlay(im, rr), f"{name}: hands + devices"), label_strip(rr.frame, "camera output")]
    for it in rr.interactions:
        rows.append({"image": name, "hand": it.hand.handedness, "device": it.device.label,
                     "device score": round(it.device.score, 2), "confidence": round(it.confidence, 2),
                     "evidence": ", ".join(it.reasons)})
show(side_by_side(*tiles[:4], height=380), save="stage_pin_a.png")
show(side_by_side(*tiles[4:], height=480), save="stage_pin_b.png", width=11)
pd.DataFrame(rows)
```





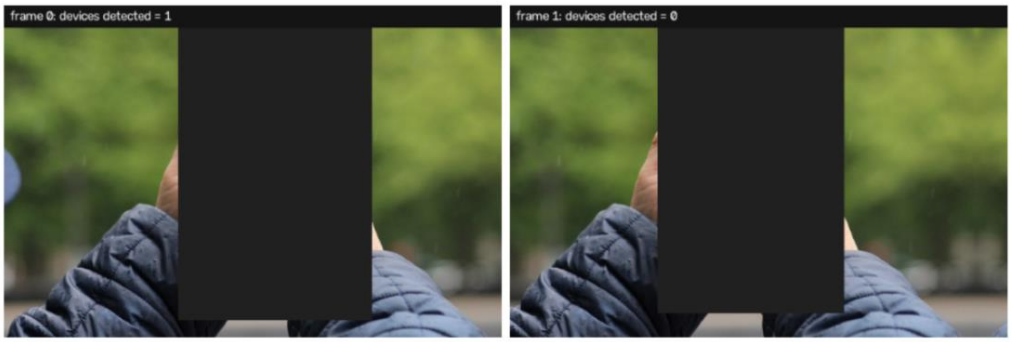
Out[6]:

	image	hand	device	device score	confidence	evidence
0	ua_00.jpg	Right	cell phone	0.72	0.6	fingertip_on_device
1	phone_00.jpg	Left	cell phone	0.64	0.8	fingertip_on_device, thumb_on_held_phone

Figure 6. Notebook: PIN guard. Left of each pair: hands and devices found; right: the only output that leaves the camera.

In [7]:
```
class FirstFrameOnly:
    def __init__(self, inner): self.inner, self.n = inner, 0
    def __call__(self, *a, **k):
        self.n += 1
        return self.inner(*a, **k) if self.n == 1 else []
    def close(self): self.inner.close()

base = load("ua_00.jpg")
occ = PrivacyPipeline(PrivacyConfig(body_privacy=False))
occ.devices = FirstFrameOnly(occ.devices)
M = np.float32([[1, 0, -40], [0, 1, 0]])
seq = [base, cv2.warpAffine(base, M, (base.shape[1], base.shape[0]), borderMode=cv2.BORDER_REFLECT)]
rows, outs = [], []
for i, f in enumerate(seq):
    rr = occ.process(f.copy())
    outs.append(label_strip(rr.frame, f"frame {i}: devices detected = {len(rr.devices)}"))
    rows += [{"frame": i, "device source": it.device.source, "evidence": ", ".join(it.reasons)} for it in rr.interactions]
show(side_by_side(*outs, height=330), save="stage_occlusion.png")
pd.DataFrame(rows)
```



Out[7]:

	frame	device source	evidence
0	0	devices	fingertip_on_device
1	1	device_memory	device_occluded_by_hand

Figure 7. Notebook: the device detector is forced to miss after frame 0, and the device mask is carried with the hand.

In [8]:
```
from privacy_cctv.masking import apply_masks
im = load("ua_00.jpg")
box = Box(200, 40, 470, 345)
x1, y1, x2, y2 = box.as_int()
styles = {"original (never leaves camera)": None, "gaussian blur (NOT used)": "blur",
          "pixelate (body regions)": "pixelate", "solid fill (PIN regions)": "solid"}
tiles, rows = [], []
for title, style in styles.items():
    f = im.copy()
    if style == "blur":
        f[y1:y2, x1:x2] = cv2.GaussianBlur(f[y1:y2, x1:x2], (31, 31), 8)
    elif style:
        apply_masks(f, [box], cells=6, style=style)
    crop = f[y1:y2, x1:x2]
    rows.append({"method": title, "distinct colours left": len(np.unique(crop.reshape(-1, 3), axis=0)),
                 "reversible": "-" if style is None else ("partly (deconvolution)" if style == "blur" else "no")})
    tiles.append(label_strip(crop, title))
show(side_by_side(*tiles, height=300), save="masking_styles.png")
pd.DataFrame(rows)
```



Out[8]:

	method	distinct colours left	reversible
0	original (never leaves camera)	49500	-
1	gaussian blur (NOT used)	36661	partly (deconvolution)
2	pixelate (body regions)	30	no
3	solid fill (PIN regions)	1	no

Figure 8. Notebook: masking styles. Blur keeps thousands of colours and is partly reversible; the pipeline's pixelation and solid fill keep 30 and 1.



Figure 9. Notebook: fail-closed. A simulated NPU fault withholds the whole frame.

8. Results and Applications

8.1 Protection and over-masking

Metric	Result
Set A: sensitive regions protected (>=90% masked)	4/8
Set A: mean coverage of sensitive regions	58.1%
Set A: over-mask (share of frame outside truth)	3.6%
Set B: false alarms (device, no PIN entry)	1/26
Set C: false alarms (clothed people)	0/21
Sets B+C: mean share of frame masked	0.04%

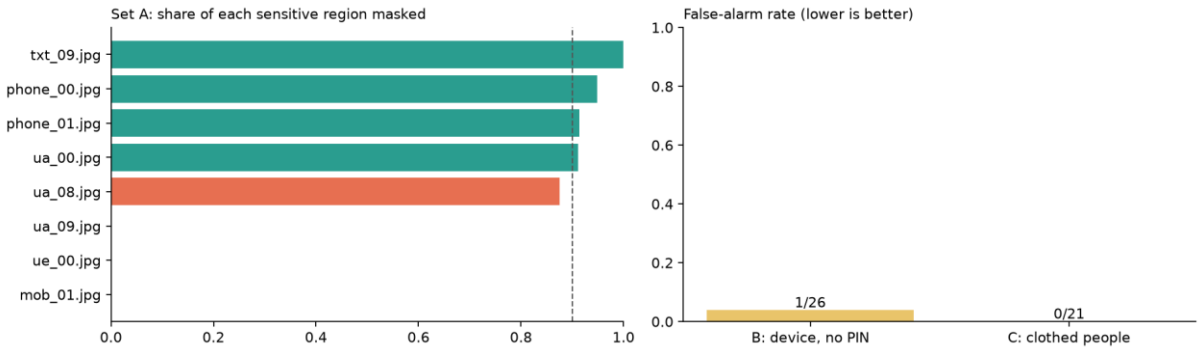


Figure 10. Left: share of each Set A sensitive region masked (dashed line: 90 % protection bar). Right: false-alarm rate on Sets B and C.

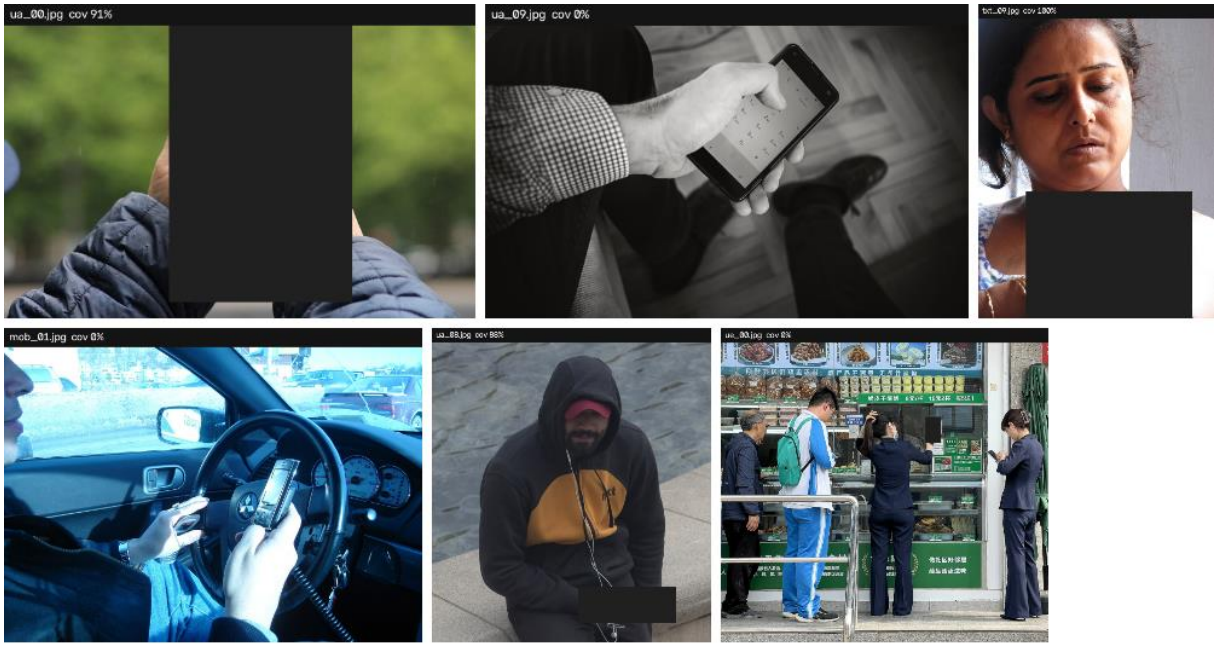


Figure 11. Camera output for Set A images (coverage in each title).

Looking at the misses: all Set A failures are detection failures, not fusion errors. In one image neither the hand nor the phone was found (a driver photographed through a windscreen). In another the phone was found, but the hand model missed a hand coming out of a sleeve cuff. And in the most CCTV-like image (customers at a shop counter, phone about 20 pixels across) the hand and phone were too small even for the zoom passes. Whenever both the hand and the device were detected, the region was protected, or very nearly. The only remaining Set B false alarm is a vendor holding a QR-code stand, which the COCO detector reads as a phone.

8.2 Policies and profiles

policy	Set A protected	Set A mean coverage	Set B false alarms
interaction-gated (default)	4/8	58.1%	1/26
default without zoom passes	4/8	47.2%	0/26
mask every device (bank)	5/8	70.6%	16/26

Ablation: switching the guided zoom passes off lowers Set A mean coverage from 58.1% to 47.2%, while Set B false alarms go from 1/26 to 0/26. The zoom passes find small hands and phones the full-frame models miss, and the one false alarm they add is the QR-code stand.

If you mask every device (the bank profile), you catch the PIN entries whose hand was never detected, but you also mask idle devices. That's fine for an ATM or PIN-pad zone.

profile	set C images	false alarms	mean share of frame masked
retail	21	0	0.00%
hostel	21	0	0.00%

8.3 Temporal hold

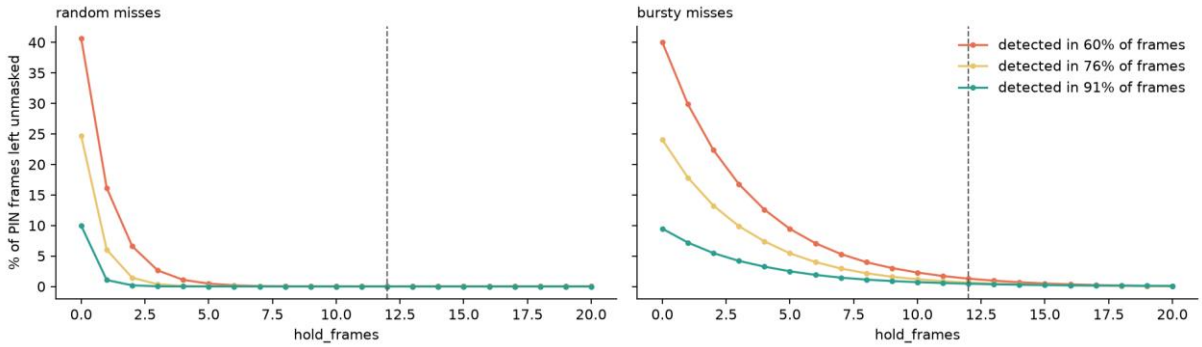


Figure 12. Share of PIN-entry frames left unmasked against the hold length, for three per-frame detection rates and two miss patterns (simulation).

misses	detection rate	leak, no hold	leak, hold=12
random misses	59%	40.7%	0.01%
bursty misses	60%	40.0%	1.28%
random misses	75%	24.7%	0.00%
bursty misses	76%	24.0%	0.63%
random misses	90%	10.0%	0.00%
bursty misses	91%	9.5%	0.45%

Without a hold, the leak rate is the same as the detector's miss rate. With a 12 frame hold (0.8s at 15fps), random misses are pretty much completely filtered out. Bursty misses, where the hand hides the device for a few frames, still leak a bit, but that's exactly what the occlusion memory is aimed at.

8.4 Speed

Stage	Mean ms / frame
prep	1.3
pose	41.7
body_parts	44.6
devices	55.1
hands	57.7
hands_roi	3.9
devices_roi	46.4
fuse	0.4
mask	0.1
total (mean / p95)	139.1 / 159.9
frames per second	7.2

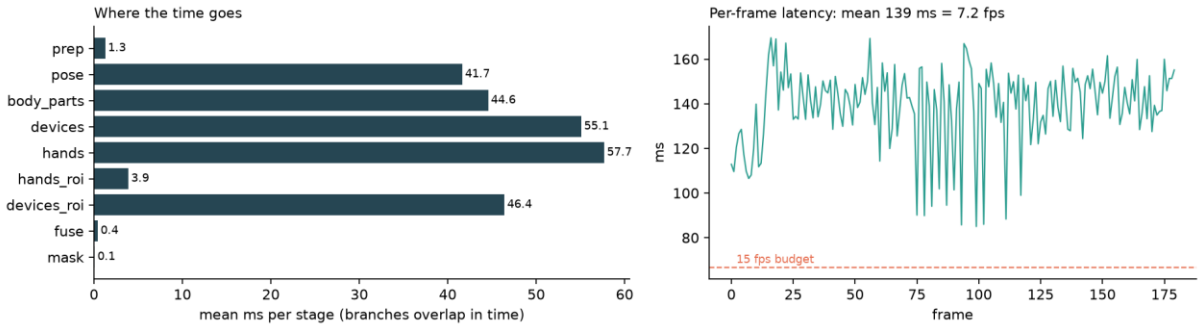


Figure 13. Per-stage cost and per-frame latency on a simulated 640x480 camera pan (laptop CPU, video mode).

Configuration	Mean ms	p95 ms	fps
serial, 1 zoom crop	197.5	221.0	5.1
parallel, 1 zoom crop (live)	139.1	159.9	7.2
parallel, zoom off	95.5	105.3	10.5

Running the three independent branches (pose -> body parts, device detector, hand tracker) in parallel threads cuts the mean latency from 197.5 ms to 139.1 ms. The zoom passes add about 44 ms on this busy scene (several small hands), and less on quiet scenes.

On the laptop CPU the full pipeline runs at 7.2 fps on a busy scene. That is usable for a demonstration, but below the 15 fps of typical CCTV recording. The four networks dominate the cost. They total about 30 MB and are standard architectures (BlazePose, YOLOv8n, EfficientDet-Lite, BlazePalm) that commonly run in real time on edge accelerators after int8 quantisation, so meeting real-time speed on camera hardware is a deployment step, not a change of method.

8.5 Applications

- **Hostels, PGs and dormitories:** corridor cameras that keep safety value without recording bodies.
- **Retail billing counters and UPI payments:** masking customers' PIN entry on phones and POS terminals.
- **Banks and ATMs:** strict profile masking PIN pads continuously.
- **Hospitals and elder care:** patient dignity in wards and corridors.
- **Gyms, pools and changing-room approaches:** areas where partial undress is expected.
- **Any camera that exports footage:** police requests, insurance claims and social-media leaks carry no sensitive regions.

9. Advantages and Limitations

Advantages

- **Privacy by construction:** unmasked footage never exists outside the camera, so it cannot leak or be hacked.
- **Evidence preserved:** only regions without forensic value are removed; faces, actions and the scene remain.
- **Targeted:** sets B and C show near-zero masking of idle devices and clothed people.
- **Fail-safe:** model errors withhold frames; flicker is absorbed by the hold.

- **Configurable:** hostel, retail and bank profiles choose what counts as sensitive.
- **Low cost:** pre-trained models, CPU-only prototype, no cloud processing or extra bandwidth.

Limitations

- **Small evaluation set:** 55 labelled still images (8 positives). It shows behaviour, not a statistically strong accuracy estimate.
- **Detector recall bounds protection:** if the pre-trained models miss a hand or a phone, nothing is masked (3 of 8 positives). Small, distant subjects are the weak case.
- **Body pipeline recall not measured on nudity:** for ethical reasons no explicit images were used; only the false-alarm side was measured, on clothed people.
- **No dedicated POS class:** COCO has no POS-terminal class. The 'cell phone' class catches many terminals, but not all.
- **Speed:** 7.2 fps on a laptop CPU, below real-time CCTV rates without an accelerator.
- **Pose model:** the lite model has trouble seeing far-away people and sometimes builds 'people' out of hands, so plausibility checks were added.
- **Untested conditions:** night-time IR footage, crowds, fisheye lenses and deliberate evasion.

10. Ethical Considerations

- **Privacy and law.** In line with purpose-limitation and data-minimisation principles under India's Digital Personal Data Protection Act, 2023 [8] and the recognition of privacy as a fundamental right (Puttaswamy, 2017 [9]), the risk of unauthorised image capture of a private part of a person (offence under Section 66E of IT Act [10] and voyeurism offence under Section 77 of Bharatiya Nyaya Sanhita [11]) can be mitigated for camera owners by source masking.
- **Fairness and bias.** Pose, hand and body-part models can perform differently across skin tones, body types, clothing styles and disabilities. A miss is a privacy failure for that person, so the system needs per-group testing on consented, diverse data before deployment. Masking is offered for all genders (male chest masking is configurable per site).
- **Security.** The masking stage has to be part of signed firmware, and the debug and raw-preview modes have to be removed from production builds, or an attacker could disable it. The prototype's model-view overlays exist only in development.
- **No function creep.** The pipeline only detects body parts and devices in order to remove them. No identification, no face recognition, no tracking of individuals, and its output shouldn't be reused for that.
- **Transparency and consent.** Signage should say that the camera masks sensitive areas. A count of masked regions can be kept in the metadata, so auditors can check that the system was running without seeing the actual content.
- **Evidence integrity.** Masks are always permanent. The answer is no if an investigator wants a masked area. This trade-off should be documented for the legal team at the site.
- **Responsible data handling in this project.** No nudity and no real PINs were collected. Sample images are openly licensed and credited. Images that turned out to contain nudity were deleted.

11. Future Scope

- **Dedicated POS / PIN-pad detector** fine-tuned (e.g. with MediaPipe Model Maker or YOLO) on consented counter recordings.
- **Multi-person pose** (YOLO-pose or RTMPose) for crowded corridors and small, distant people.
- **Edge deployment:** int8 quantisation and compilation for camera NPUs (e.g. Rockchip RV1126, Ambarella CVflow, Hailo-8L), with the masking stage inside the ISP-to-encoder path.
- **Temporal models:** classify PIN-typing from landmark sequences (TCN/LSTM) instead of single-frame rules.
- **Segmentation masks** instead of boxes, to hide even less of the scene.
- **Benchmark:** a consented, diverse, frame-labelled video dataset of counters and corridors, with per-group fairness metrics.
- **Tamper detection:** alert when the camera is covered, re-aimed or its firmware altered.

12. Conclusion

This project demonstrates that the existing AI hardware in modern CCTV cameras can be used to increase privacy instead of decreasing it. By utilizing pose estimation and a pre-trained body part detector, object detector, and hand tracking, the prototype hides private body parts and PIN entry in the camera itself (before any video is recorded or transferred) for good, yet leaves the rest of the scene available for evidence.

On labelled test images it protected 4/8 PIN-entry regions, with 1/26 false alarms on idle devices and 0/21 on clothed people. A simulation showed that a 12-frame hold cuts the share of leaked PIN frames from 10-40 % (the detector's miss rate) to under 2 %, even when misses come in bursts. Every miss traced back to a hand or device the pre-trained detectors did not find, so the clear next steps are better small-object detection, a dedicated POS model and deployment on a camera NPU for real-time speed. The architecture (mask at the source, mask early and release late, fail closed) holds regardless of which detectors are plugged in.

References

[1] C. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," arXiv:1906.08172, 2019.

[2] V. Bazarevsky et al., "BlazePose: On-device Real-time Body Pose Tracking," arXiv:2006.10204, 2020.

[3] F. Zhang et al., "MediaPipe Hands: On-device Real-time Hand Tracking," arXiv:2006.10214, 2020.

[4] M. Tan, R. Pang, Q. V. Le, "EfficientDet: Scalable and Efficient Object Detection," CVPR, 2020.

[5] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," ECCV, 2014.

[6] G. Jocher et al., Ultralytics YOLOv8, 2023. https://github.com/ultralytics/ultralytics

[7] P. Bedapudi, NudeNet: lightweight nudity detection (v3). https://github.com/notAI-tech/NudeNet

[8] Government of India, The Digital Personal Data Protection Act, 2023.

[9] Justice K. S. Puttaswamy (Retd.) v. Union of India, (2017) 10 SCC 1.

[10] Government of India, The Information Technology Act, 2000, Section 66E.

[11] Government of India, The Bharatiya Nyaya Sanhita, 2023, Section 77.

[12] M. Antonakakis et al., "Understanding the Mirai Botnet," USENIX Security Symposium, 2017.

[13] Wikimedia Commons: test images and licences listed in data/samples/sources.json. https://commons.wikimedia.org

[14] Google, MediaPipe Tasks model cards (Pose Landmarker, Hand Landmarker, Object Detector). https://ai.google.dev/edge/mediapipe
[15] A. Cavoukian, "Privacy by Design: The 7 Foundational Principles," Information and Privacy Commissioner of Ontario, 2009.
[16] ONNX Runtime (https://onnxruntime.ai) and OpenCV (https://opencv.org).

Appendix A: Jupyter notebook screenshots

Screenshots of the executed notebook (notebooks/privacy_cctv_demo.ipynb), in order. Cells were run top to bottom on the same laptop used for the timing results.

Privacy-by-Design CCTV: prototype notebook

Leon Kaushik Deka · Third Semester AIML Mini Project · October 2026

An AI pipeline that runs **inside the camera**, so a frame is masked before it can be stored or streamed.

Pipeline	Models	What gets masked
Body privacy	MediaPipe **Pose Landmarker** (33 keypoints) + **NudeNet v3** body-part detector (YOLOv8n, 18 classes, ONNX)	nipples, buttocks, genitalia
PIN guard	**EfficientDet-Lite0** (COCO: *cell phone, remote*) + MediaPipe **Hand Landmarker** (21 keypoints)	device screen/keypad **and** the hand entering the PIN

Masks are written into the frame buffer itself (permanent), PIN regions are filled solid, body regions are pixelated, and any model fault blanks the whole frame (fail-closed).

This notebook walks through each stage on openly licensed images, then measures the three things the prototype is tested on: **how well it masks sensitive areas**, **how often it masks unnecessary areas**, and **how fast it runs**.

In [1]:
```
import os, sys, json, time, platform, warnings
os.environ["GLOG_minloglevel"] = "3"
warnings.filterwarnings("ignore")
from pathlib import Path

ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
sys.path.insert(0, str(ROOT))

import cv2, numpy as np, pandas as pd, matplotlib.pyplot as plt
import mediapipe, onnxruntime
from privacy_cctv import PrivacyPipeline, PrivacyConfig, PROFILES, Box
from privacy_cctv.config import PART_REGION
from privacy_cctv.viz import overlay, side_by_side, label_strip
from privacy_cctv.metrics import score_image, latency_summary, leak_rate

plt.rcParams.update({"figure.dpi": 110, "axes.spines.top": False, "axes.spines.right": False})
FIG = ROOT / "docs" / "figures"; FIG.mkdir(parents=True, exist_ok=True)
SAMPLES = ROOT / "data" / "samples"
RESULTS = {}

def load(name, max_side=640):
    img = cv2.imread(str(SAMPLES / name))
    s = max_side / max(img.shape[:2])
    return cv2.resize(img, (round(img.shape[1] * s), round(img.shape[0] * s)), interpolation=cv2.INTER_AREA)

def show(img, title=None, width=13, save=None):
    if save: cv2.imwrite(str(FIG / save), img)
    plt.figure(figsize=(width, width * img.shape[0] / img.shape[1]))
    plt.imshow(img[:, :, ::-1]); plt.axis("off")
    if title: plt.title(title, loc="left", fontsize=11)
    plt.show()

pd.DataFrame({
    "component": ["Python", "OpenCV", "MediaPipe", "ONNX Runtime", "CPU"],
    "version": [platform.python_version(), cv2.__version__, mediapipe.__version__, onnxruntime.__version__,
                platform.processor() or platform.machine()],
})
```

Out[1]:

	component	version
0	Python	3.12.12
1	OpenCV	5.0.0
2	MediaPipe	1.2.0
3	ONNX Runtime	1.31.1
4	CPU	AMD64 Family 25 Model 80 Stepping 0, AuthenticAMD

Notebook cell 1

1. Data

No footage of real people undressing or entering real PINs was collected (that would itself breach the privacy this project protects). The prototype uses:

- **Pre-trained models:** trained by their authors on large datasets (COCO for EfficientDet; Google's pose and hand datasets for MediaPipe; the NudeNet dataset for body parts).
- **Openly licensed test images from Wikimedia Commons:** shops, corridors, payment terminals and people using phones, each with its licence recorded in `data/samples/sources.json`.
- **A hand-labelled evaluation set** (`data/eval/labels.json`) split into three groups.

In [2]:
```
sources = json.loads((SAMPLES / "sources.json").read_text())
labels = json.loads((ROOT / "data" / "eval" / "labels.json").read_text())
images = labels["images"]

src = pd.DataFrame.from_dict(sources, orient="index")
lic = src["license"].replace("", "see page").value_counts().rename_axis("licence").reset_index(name="images")

sets = pd.Series({k: v["set"] for k, v in images.items()}).value_counts().sort_index()
desc = {"A": "hand operating a phone / POS (must be masked)",
        "B": "device visible, nobody entering a PIN (must NOT be masked)",
        "C": "ordinary clothed people (must NOT be masked)"}
eval_table = pd.DataFrame({"set": sets.index, "images": sets.values, "meaning": [desc[s] for s in sets.index]})
display(eval_table); display(lic)
```

	set	images	meaning
0	A	8	hand operating a phone / POS (must be masked)
1	B	26	device visible, nobody entering a PIN (must NO...
2	C	21	ordinary clothed people (must NOT be masked)

	licence	images
0	CC BY-SA 4.0	24
1	CC BY-SA 3.0	14
2	CC BY 2.0	12
3	CC0	11
4	CC BY 4.0	11
5	CC BY-SA 2.0	6
6	CC BY 3.0	3
7	OGL v1.0	1
8	Public domain	1

Notebook cell 3

2. Body privacy: pose estimation

The pose model returns 33 keypoints per person. From the shoulders and hips the pipeline derives two **anatomical regions** (chest and pelvis). They are used to

1. **crop** each person and run the body-part detector at full resolution on the crop (people far down a corridor are small),
2. **gate** detections: a part found inside the region its class belongs to is accepted at a lower score (0.25) than one found anywhere else (0.45), and
3. **anchor** a detected part to the hips, so the mask follows the body if the detector misses a frame.

In [3]:
```
pipe = PrivacyPipeline(PrivacyConfig())
img = load("phone_06.jpg")
r = pipe.process(img.copy())
vis = overlay(img, r)
show(label_strip(vis, "Pose keypoints -> chest and pelvis regions"), width=6, save="stage_pose.png")
p = r.people[0]
pd.DataFrame([{"region": k, "x1": round(b.x1), "y1": round(b.y1), "x2": round(b.x2), "y2": round(b.y2)}
              for k, b in p.regions().items()]).assign(torso_px=round(p.torso_len))
```



Out[3]:

	region	x1	y1	x2	y2	torso_px
0	chest	160	213	305	334	221
1	pelvis	139	377	275	553	221

Notebook cell 5

3. Body privacy: body-part detector and pose gating

NudeNet detects 18 body-part classes, both *exposed* and *covered*. Only the exposed intimate classes are sensitive in the default profile. Two vetoes remove the detector's typical false positives: a detection lying mostly on a tracked **hand** is rejected (it confuses fingers with genitalia), and on an upright person a pelvis-class part **above the shoulders** or a chest-class part above the nose is anatomically impossible. On a fully clothed person the detector fires on covered classes, and the pipeline correctly masks nothing:

In [4]:
```
parts = pipe.body.detect_parts(img, list(r.people))
regions = p.regions()
chosen = {id(d) for d, _ in pipe.body.select(parts, list(r.people))}
rows = []
for d in sorted(parts, key=lambda d: -d.score):
    key = PART_REGION.get(d.label)
    inside = bool(key and regions[key].contains(*d.center))
    rows.append({"class": d.label, "score": round(d.score, 2), "expected region": key or "-",
                 "inside region": inside, "sensitive class": d.label in pipe.cfg.sensitive_parts,
                 "masked": id(d) in chosen})
pd.DataFrame(rows)
```

Out[4]:

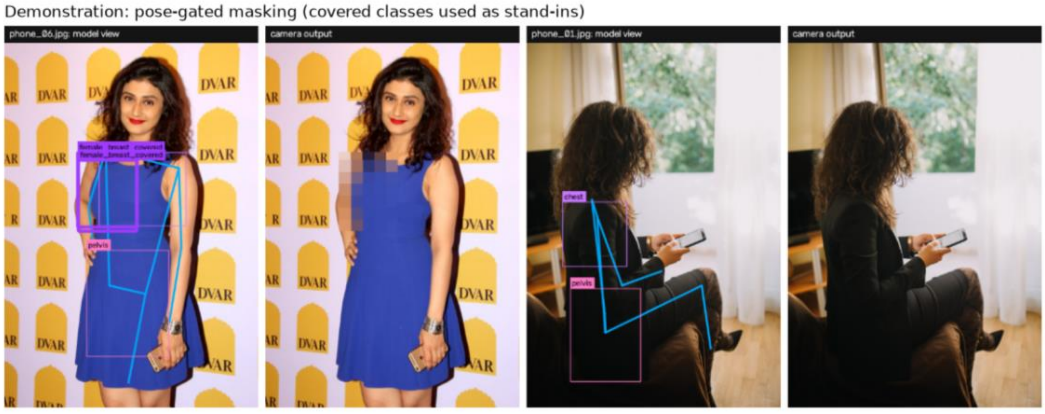
	class	score	expected region	inside region	sensitive class	masked
0	FACE_FEMALE	0.88	-	False	False	False
1	FACE_FEMALE	0.75	-	False	False	False
2	FEMALE_BREAST_COVERED	0.65	-	False	False	False
3	FEMALE_BREAST_COVERED	0.65	-	False	False	False
4	FEMALE_BREAST_COVERED	0.44	-	False	False	False
5	FEMALE_BREAST_COVERED	0.42	-	False	False	False
6	FEMALE_BREAST_COVERED	0.35	-	False	False	False
7	ARMPITS_EXPOSED	0.31	-	False	False	False
8	ARMPITS_EXPOSED	0.31	-	False	False	False
9	BELLY_COVERED	0.25	-	False	False	False
10	FEET_COVERED	0.25	-	False	False	False

Notebook cell 7

To show **where** the masks go without using any explicit imagery, the next cell temporarily treats the *covered* chest / pelvis classes as sensitive (this is what the stricter **hostel** profile does for underwear). Left: what the models see. Right: the only output that leaves the camera.

In [5]:
```
demo_cfg = PrivacyConfig(pin_privacy=False,
                         sensitive_parts=("FEMALE_BREAST_COVERED", "BUTTOCKS_COVERED", "FEMALE_GENITALIA_COVERED"))
demo = PrivacyPipeline(demo_cfg)
tiles = []
for name in ["phone_06.jpg", "phone_01.jpg"]:
    im = load(name); demo.reset()
    rr = demo.process(im.copy())
    tiles += [label_strip(overlay(im, rr), f"{name}: model view"), label_strip(rr.frame, "camera output")]
show(side_by_side(*tiles, height=420), "Demonstration: pose-gated masking (covered classes used as stand-ins)",
     save="stage_body_mask_demo.png")
```

Demonstration: pose-gated masking (covered classes used as stand-ins)



Notebook cell 9

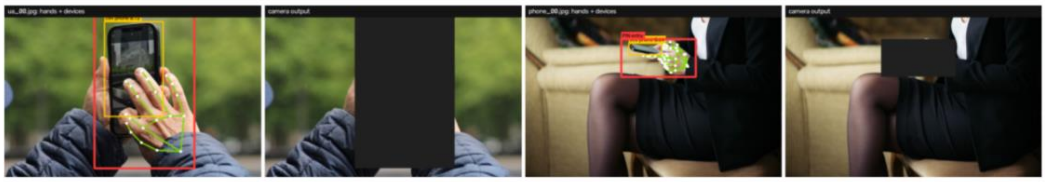
4. PIN guard: device detection and hand tracking

EfficientDet-Lite0 finds phones; the COCO *cell phone* class also fires on POS handsets and PIN pads. The hand model gives 21 keypoints per hand. A hand is treated as **entering a PIN** when a fingertip (index, middle or thumb) lands on the face of a device. Pointing with an extended index finger and tapping raise the confidence.

Hands and phones at a counter are small in a CCTV frame, so three **zoom passes** look again at full model resolution, each guided by what was already found: device → crop for the hand, pose wrist → crop for the hand, hand → crop for the device. A device held against a head (pose head or detected face) is a **phone call**, not PIN entry, and is ignored.

Mask = union of hand box and device box, expanded by 25 %, **filled solid** (a pixelated finger over a keypad can still reveal which row is pressed).

In [6]:
```
pin_pipe = PrivacyPipeline(PrivacyConfig(body_privacy=False))
tiles, rows = [], []
for name in ["ua_00.jpg", "phone_00.jpg", "ue_00.jpg"]:
    im = load(name); pin_pipe.reset()
    rr = pin_pipe.process(im.copy())
    tiles += [label_strip(overlay(im, rr), f"{name}: hands + devices"), label_strip(rr.frame, "camera output")]
    for it in rr.interactions:
        rows.append({"image": name, "hand": it.hand.handedness, "device": it.device.label,
                     "device score": round(it.device.score, 2), "confidence": round(it.confidence, 2),
                     "evidence": ", ".join(it.reasons)})
show(side_by_side(*tiles[:4], height=380), save="stage_pin_a.png")
show(side_by_side(*tiles[4:], height=480), save="stage_pin_b.png", width=11)
pd.DataFrame(rows)
```





Out[6]:

	image	hand	device	device score	confidence	evidence
0	ua_00.jpg	Right	cell phone	0.72	0.6	fingertip_on_device
1	phone_00.jpg	Left	cell phone	0.64	0.8	fingertip_on_device, thumb_on_held_phone

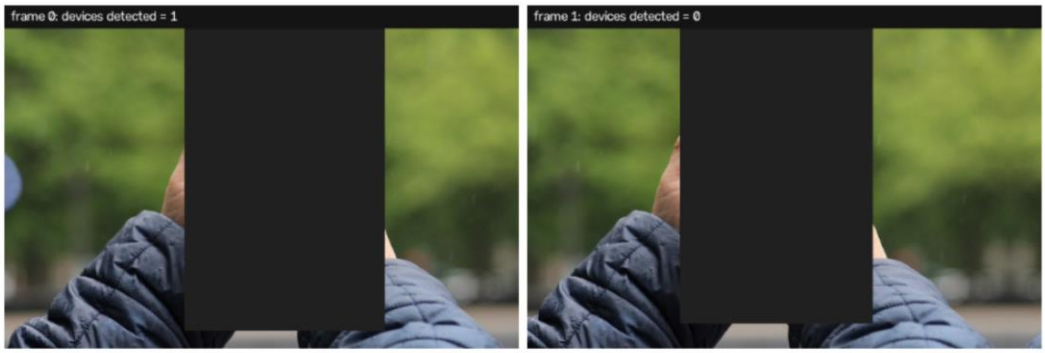
Notebook cell 11

Occlusion: the hand hides the device

While a PIN is typed the hand often covers the phone, and the object detector loses it. The guard stores the last device box relative to the wrist and carries it with the hand. Below, the device detector is forced to return nothing after the first frame and the hand moves 40 px; the mask still follows the hand.

In [7]:
```
class FirstFrameOnly:
    def __init__(self, inner): self.inner, self.n = inner, 0
    def __call__(self, *a, **k):
        self.n += 1
        return self.inner(*a, **k) if self.n == 1 else []
    def close(self): self.inner.close()

base = load("ua_00.jpg")
occ = PrivacyPipeline(PrivacyConfig(body_privacy=False))
occ.devices = FirstFrameOnly(occ.devices)
M = np.float32([[1, 0, -40], [0, 1, 0]])
seq = [base, cv2.warpAffine(base, M, (base.shape[1], base.shape[0]), borderMode=cv2.BORDER_REFLECT)]
rows, outs = [], []
for i, f in enumerate(seq):
    rr = occ.process(f.copy())
    outs.append(label_strip(rr.frame, f"frame {i}: devices detected = {len(rr.devices)}"))
    rows += [{"frame": i, "device source": it.device.source, "evidence": ", ".join(it.reasons)} for it in rr.interactions]
show(side_by_side(*outs, height=330), save="stage_occlusion.png")
pd.DataFrame(rows)
```



Out[7]:

	frame	device source	evidence
0	0	devices	fingertip_on_device
1	1	device_memory	device_occluded_by_hand

Notebook cell 13

5. Masking is permanent

The mask is applied to the frame buffer in place, before encoding, so no unmasked copy exists. Gaussian blur is deliberately **not** used: a known blur kernel can be partly inverted by deconvolution. Pixelation keeps only a few averaged cells; solid fill keeps nothing.

In [8]:
```
from privacy_cctv.masking import apply_masks
im = load("ua_00.jpg")
box = Box(200, 40, 470, 345)
x1, y1, x2, y2 = box.as_int()
styles = {"original (never leaves camera)": None, "gaussian blur (NOT used)": "blur",
          "pixelate (body regions)": "pixelate", "solid fill (PIN regions)": "solid"}
tiles, rows = [], []
for title, style in styles.items():
    f = im.copy()
    if style == "blur":
        f[y1:y2, x1:x2] = cv2.GaussianBlur(f[y1:y2, x1:x2], (31, 31), 8)
    elif style:
        apply_masks(f, [box], cells=6, style=style)
    crop = f[y1:y2, x1:x2]
    rows.append({"method": title, "distinct colours left": len(np.unique(crop.reshape(-1, 3), axis=0)),
                 "reversible": "-" if style is None else ("partly (deconvolution)" if style == "blur" else "no")})
    tiles.append(label_strip(crop, title))
show(side_by_side(*tiles, height=300), save="masking_styles.png")
pd.DataFrame(rows)
```



Out[8]:

	method	distinct colours left	reversible
0	original (never leaves camera)	49500	-
1	gaussian blur (NOT used)	36661	partly (deconvolution)
2	pixelate (body regions)	30	no
3	solid fill (PIN regions)	1	no

Notebook cell 15

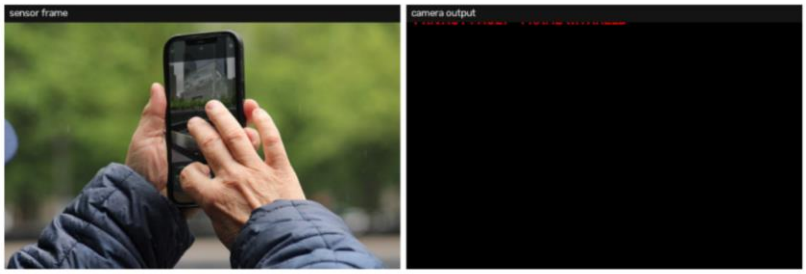
6. Fail-closed

If any model raises (corrupt frame, NPU fault, out of memory) the frame is **withheld**, never passed through unmasked.

In [9]:
```
class BrokenModel:
    def __call__(self, *a, **k): raise RuntimeError("simulated NPU fault")
    def close(self): pass

faulty = PrivacyPipeline(PrivacyConfig(body_privacy=False))
faulty.hands = BrokenModel()
im = load("ua_00.jpg")
rr = faulty.process(im.copy())
print("fault:", rr.fault, "| max pixel value in output:", int(rr.frame[40:].max()))
show(side_by_side(label_strip(im, "sensor frame"), label_strip(rr.frame, "camera output")), width=10,
     save="failclosed.png")
```

```
fault: RuntimeError: simulated NPU fault | max pixel value in output: 0
```



Notebook cell 17

7. Evaluation on the labelled set

Every image is processed independently with the full pipeline (default profile: body privacy + PIN guard).

- **Protected:** a sensitive region counts as protected if at least 90 % of its pixels are masked.
- **False alarm:** an image from set B or C with *anything* masked.
- **Over-mask:** masked pixels outside every sensitive region, as a share of the frame.

In [10]:
```
full = PrivacyPipeline(PrivacyConfig())
scores, outputs = [], {}
for name, lab in images.items():
    im = load(name); full.reset()
    rr = full.process(im.copy())
    truth = [Box(*b) for b in lab.get("sensitive", [])]
    ignore = [Box(*b) for b in lab.get("ignore", [])]
    s = score_image(name, im.shape, list(rr.regions), truth, ignore)
    scores.append({**s.__dict__, "set": lab["set"], "ms": rr.timings_ms["total"]})
    outputs[name] = (im, rr)
df = pd.DataFrame(scores)

def summarise(d):
    a, bc = d[d.set == "A"], d[d.set != "A"]
    return pd.DataFrame([
        {"metric": "Set A: sensitive regions protected (>=90% masked)", "value": f"{a.protected.sum()}/{a.gt_regions.sum()}"},
        {"metric": "Set A: mean coverage of sensitive regions", "value": f"{a.mean_coverage.mean():.1%}"},
        {"metric": "Set A: over-mask (share of frame outside truth)", "value": f"{a.over_mask_frac.mean():.1%}"},
        {"metric": "Set B: false alarms (device, no PIN entry)", "value": f"{d[d.set=='B'].false_alarm.sum()}/{(d.set=='B').sum()}"},
        {"metric": "Set C: false alarms (clothed people)", "value": f"{d[d.set=='C'].false_alarm.sum()}/{(d.set=='C').sum()}"},
        {"metric": "Sets B+C: mean share of frame masked", "value": f"{bc.masked_frac.mean():.2%}"},
    ])

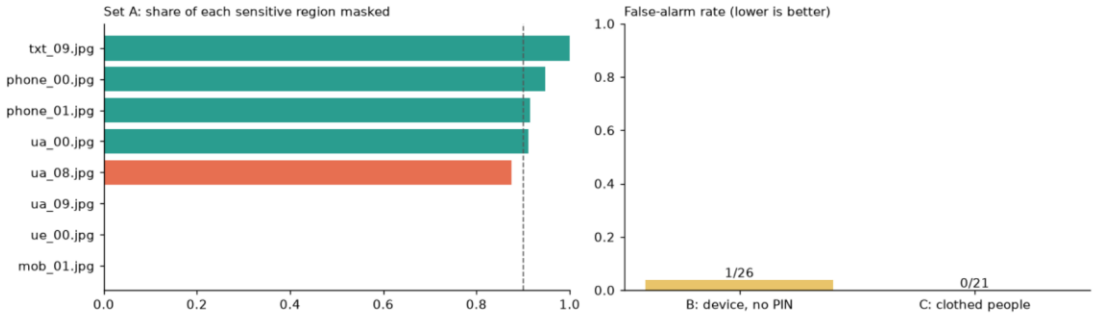
summary = summarise(df)
RESULTS["eval"] = {r.metric: r.value for r in summary.itertuples()}
summary
```

Out[10]:

	metric	value
0	Set A: sensitive regions protected (>=90% masked)	4/8
1	Set A: mean coverage of sensitive regions	58.1%
2	Set A: over-mask (share of frame outside truth)	3.6%
3	Set B: false alarms (device, no PIN entry)	1/26
4	Set C: false alarms (clothed people)	0/21
5	Sets B+C: mean share of frame masked	0.04%

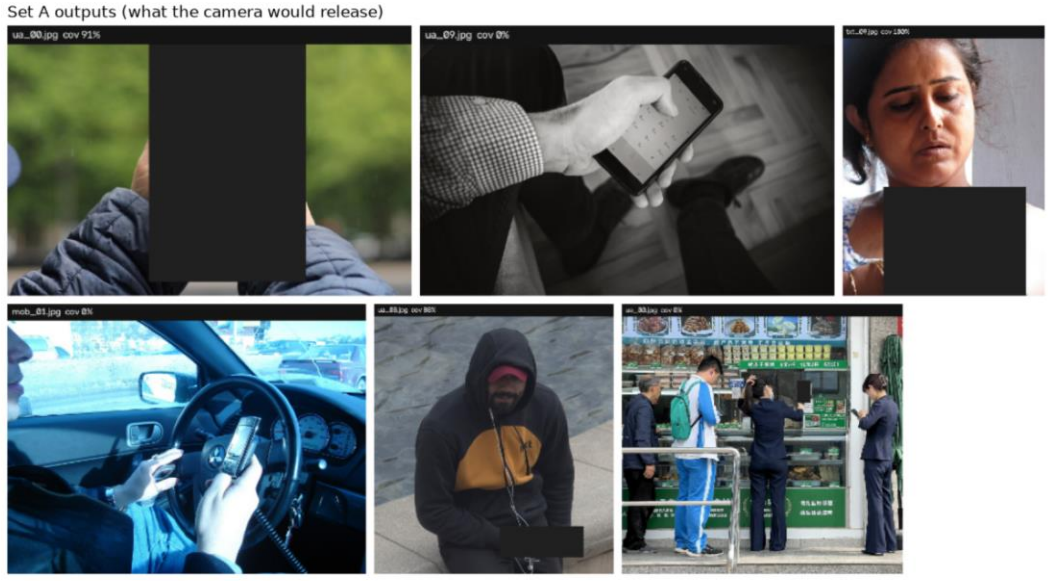
Notebook cell 19

In [11]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 3.6))
a = df[df.set == "A"].sort_values("mean_coverage")
axes[0].barh(a.name, a.mean_coverage, color=["#2a9d8f" if c >= 0.9 else "#e76f51" for c in a.mean_coverage])
axes[0].axvline(0.9, ls="--", c="#555", lw=1); axes[0].set_xlim(0, 1)
axes[0].set_title("Set A: share of each sensitive region masked", loc="left", fontsize=10)
fa = df.groupby("set").agg(images=("name", "size"), alarms=("false_alarm", "sum"))
fa = fa.loc[["B", "C"]]
axes[1].bar(["B: device, no PIN", "C: clothed people"], fa.alarms / fa.images, color="#e9c46a")
for i, (n, k) in enumerate(zip(fa.images, fa.alarms)):
    axes[1].text(i, k / n + 0.01, f"{k}/{n}", ha="center")
axes[1].set_ylim(0, 1); axes[1].set_title("False-alarm rate (lower is better)", loc="left", fontsize=10)
plt.tight_layout(); plt.savefig(FIG / "eval_bars.png", dpi=160, bbox_inches="tight"); plt.show()
```



Notebook cell 20

In [12]:
```
tiles = [label_strip(outputs[n][1].frame, f"{n}  cov {df.set_index('name').mean_coverage[n]:.0%}")
         for n in ["ua_00.jpg", "ua_09.jpg", "txt_09.jpg", "mob_01.jpg", "ua_08.jpg", "ue_00.jpg"]]
row1, row2 = side_by_side(*tiles[:3], height=260), side_by_side(*tiles[3:], height=260)
w = max(row1.shape[1], row2.shape[1])
pad = lambda x: cv2.copyMakeBorder(x, 0, 8, 0, w - x.shape[1], cv2.BORDER_CONSTANT, value=(255, 255, 255))
show(np.vstack([pad(row1), pad(row2)]), "Set A outputs (what the camera would release)", save="eval_gallery.png")
```



Notebook cell 21

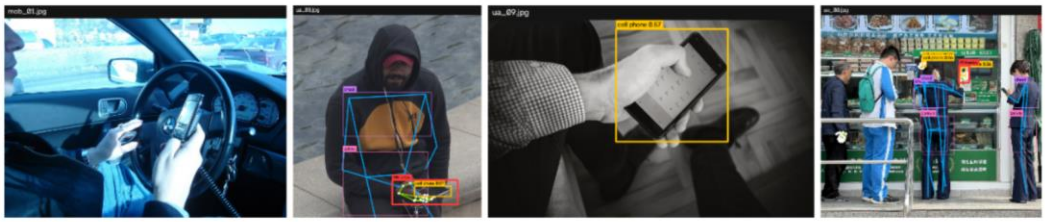
Failures and false alarms

Listing the cases that went wrong is part of the result.

In [13]:
```
bad = df[((df.set == "A") & (df.mean_coverage < 0.9)) | df.false_alarm]
cols = ["name", "set", "mean_coverage", "masked_frac", "false_alarm"]
display(bad[cols].round(3))
names = list(bad.name)[:4]
if names:
    show(side_by_side(*[label_strip(overlay(*outputs[n][:1], outputs[n][1]), n) for n in names], height=260),
         "Model view of the failures", save="eval_failures.png")
```

	name	set	mean_coverage	masked_frac	false_alarm
2	mob_01.jpg	A	0.000	0.000	False
5	ua_08.jpg	A	0.875	0.040	False
6	ua_09.jpg	A	0.000	0.000	False
7	ue_00.jpg	A	0.000	0.001	False
29	ue_01.jpg	B	NaN	0.020	True

Model view of the failures



Notebook cell 23

Profile trade-off: hostel mode on clothed people

The hostel profile also masks *covered* buttocks / genitalia (underwear in corridors). That costs false alarms on ordinary clothed people, which is why it is a per-site choice rather than the default.

In [14]:
```
rows = []
for prof in ["retail", "hostel"]:
    pp = PrivacyPipeline(PROFILES[prof])
    c_names = [n for n, l in images.items() if l["set"] == "C"]
    alarms, area = 0, []
    for n in c_names:
        im = load(n); pp.reset()
        rr = pp.process(im.copy())
        s = score_image(n, im.shape, list(rr.regions), [])
        alarms += s.false_alarm; area.append(s.masked_frac)
    rows.append({"profile": prof, "set C images": len(c_names), "false alarms": alarms,
                 "mean share of frame masked": f"{np.mean(area):.2%}"})
    pp.close()
RESULTS["profiles"] = rows
pd.DataFrame(rows)
```

Out[14]:

	profile	set C images	false alarms	mean share of frame masked
0	retail	21	0	0.00%
1	hostel	21	0	0.00%

Notebook cell 25

Policy trade-off: interaction-gated vs. mask every device

The default policy masks a device only while a hand is operating it. The stricter **bank** policy masks every detected payment device all the time (sensible for an ATM or PIN-pad zone). It recovers PIN entries where the hand was never detected, but it also masks idle devices. The middle row is an ablation with the zoom passes switched off.

In [15]:
```
rows = []
for label, cfg in [("interaction-gated (default)", PrivacyConfig()),
                   ("default without zoom passes", PrivacyConfig(zoom_crops=0)),
                   ("mask every device (bank)", PrivacyConfig(mask_all_devices=True))]:
    pp = PrivacyPipeline(cfg)
    res = {"A": [], "B": []}
    for n, l in images.items():
        if l["set"] not in res: continue
        im = load(n); pp.reset()
        rr = pp.process(im.copy())
        res[l["set"]].append(score_image(n, im.shape, list(rr.regions), [Box(*b) for b in l.get("sensitive", [])],
                                         [Box(*b) for b in l.get("ignore", [])]))
    pp.close()
    rows.append({"policy": label,
                 "Set A protected": f"{sum(s.protected for s in res['A'])}/{len(res['A'])}",
                 "Set A mean coverage": f"{np.mean([s.mean_coverage for s in res['A']]):.1%}",
                 "Set B false alarms": f"{sum(s.false_alarm for s in res['B'])}/{len(res['B'])}"})
RESULTS["policies"] = rows
pd.DataFrame(rows)
```

Out[15]:

	policy	Set A protected	Set A mean coverage	Set B false alarms
0	interaction-gated (default)	4/8	58.1%	1/26
1	default without zoom passes	4/8	47.2%	0/26
2	mask every device (bank)	5/8	70.6%	16/26

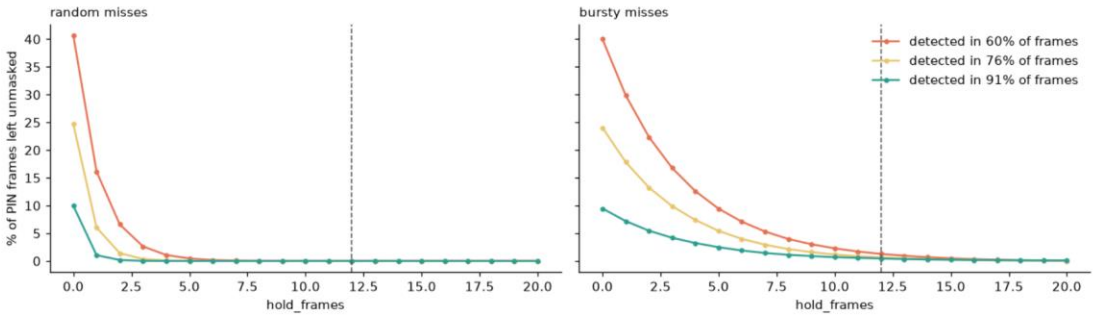
Notebook cell 27

8. Temporal hold: why masks are released late

A detector that finds the PIN pad in 80 % of frames still leaves 1 frame in 5 unmasked, which is enough to read a digit. The pipeline masks **immediately** and releases only after `hold_frames` frames without evidence. The simulation below feeds 20 000 frames of continuous PIN entry through that rule for different per-frame detection rates, with misses that are random (top) or in bursts (bottom, a hand covering the device for several frames).

In [16]:
```
rng = np.random.default_rng(7)
holds = np.arange(0, 21)
def bursty(p, n, mean_burst=4):
    out, miss = np.ones(n, bool), False
    for i in range(n):
        miss = rng.random() < (1 - 1 / mean_burst if miss else (1 - p) / (p * mean_burst))
        out[i] = not miss
    return out

fig, axes = plt.subplots(1, 2, figsize=(12, 3.6), sharey=True)
hold_rows = []
for p, col in zip([0.6, 0.75, 0.9], ["#e76f51", "#e9c46a", "#2a9d8f"]):
    for ax, kind in zip(axes, ["random misses", "bursty misses"]):
        det = rng.random(20000) < p if kind == "random misses" else bursty(p, 20000)
        leaks = [leak_rate(det, h) * 100 for h in holds]
        ax.plot(holds, leaks, marker="o", ms=3, color=col, label=f"detected in {det.mean():.0%} of frames")
        hold_rows.append({"misses": kind, "detection rate": f"{det.mean():.0%}",
                          "leak, no hold": f"{leaks[0]:.1f}%", "leak, hold=12": f"{leaks[12]:.2f}%"})
for ax, kind in zip(axes, ["random misses", "bursty misses"]):
    ax.axvline(12, ls="--", c="#555", lw=1); ax.set_xlabel("hold_frames"); ax.set_title(kind, loc="left", fontsize=10)
axes[0].set_ylabel("% of PIN frames left unmasked"); axes[1].legend(frameon=False)
plt.tight_layout(); plt.savefig(FIG / "hold_curve.png", dpi=160, bbox_inches="tight"); plt.show()
RESULTS["hold"] = hold_rows
pd.DataFrame(hold_rows)
```



Out[16]:

	misses	detection rate	leak, no hold	leak, hold=12
0	random misses	59%	40.7%	0.01%
1	bursty misses	60%	40.0%	1.28%
2	random misses	75%	24.7%	0.00%
3	bursty misses	76%	24.0%	0.63%
4	random misses	90%	10.0%	0.00%
5	bursty misses	91%	9.5%	0.45%

Notebook cell 29

9. Speed

The camera runs on a continuous stream, so the pose and hand models run in **video mode** (they track between frames instead of searching from scratch). To get a continuous 640×480 stream without filming anyone, a camera *pan* is simulated by sliding a window across a high-resolution photo of a busy shop counter (several people, hands and phones), which is close to the worst case. Timings are wall-clock on this laptop's CPU (no GPU, no NPU).

The live configuration runs the three independent branches (pose → body parts, device detector, hand tracker) **in parallel threads** and allows **one zoom crop per pass per frame**. The table compares it with running everything serially and with the zoom passes off.

In [17]:
```
big = load("ue_00.jpg", max_side=1100)
H, W = big.shape[:2]
frames = []
for i in range(180):
    t = i / 179
    x = int((W - 640) * (0.5 - 0.5 * np.cos(np.pi * 2 * t)))
    y = int((H - 480) * 0.35)
    frames.append(big[y:y + 480, x:x + 640].copy())

def bench(cfg):
    pp = PrivacyPipeline(cfg)
    for f in frames[:10]: pp.process(f.copy(), None)  # warm-up
    pp.reset()
    out = [pp.process(f.copy(), int(i * 33.3)).timings_ms for i, f in enumerate(frames)]
    pp.close()
    return pd.DataFrame(out).fillna(0)

configs = {"serial, 1 zoom crop": PrivacyConfig(video_mode=True, zoom_crops=1, parallel=False),
           "parallel, 1 zoom crop (live)": PrivacyConfig(video_mode=True, zoom_crops=1),
           "parallel, zoom off": PrivacyConfig(video_mode=True, zoom_crops=0)}
runs = {name: bench(cfg) for name, cfg in configs.items()}
abl = pd.DataFrame([{"configuration": n, **{k: round(v, 1) for k, v in latency_summary(list(d["total"])).items()}}
                    for n, d in runs.items()])
t = runs["parallel, 1 zoom crop (live)"]
stages = ["prep", "pose", "body_parts", "devices", "hands", "hands_roi", "devices_roi", "fuse", "mask"]
lat = latency_summary(list(t["total"]))
stage_tbl = pd.DataFrame({"mean ms": t[stages].mean().round(1), "p95 ms": t[stages].quantile(0.95).round(1)})
RESULTS["latency"] = {**{k: round(v, 1) for k, v in lat.items()}, "stages": stage_tbl["mean ms"].to_dict(),
                      "ablation": abl.to_dict("records")}
display(stage_tbl.T)
abl
```

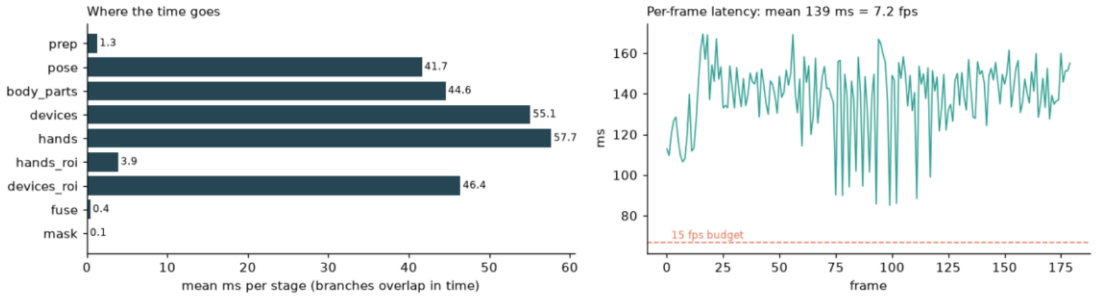
	prep	pose	body_parts	devices	hands	hands_roi	devices_roi	fuse	mask
mean ms	1.3	41.7	44.6	55.1	57.7	3.9	46.4	0.4	0.1
p95 ms	2.0	50.9	49.6	65.0	76.3	25.8	57.1	0.7	0.1

Out[17]:

	configuration	mean_ms	p50_ms	p95_ms	fps
0	serial, 1 zoom crop	197.5	200.7	221.0	5.1
1	parallel, 1 zoom crop (live)	139.1	142.6	159.9	7.2
2	parallel, zoom off	95.5	98.3	105.3	10.5

Notebook cell 31

In [18]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 3.4), gridspec_kw={"width_ratios": [1.1, 1]})
m = t[stages].mean()
axes[0].barh(m.index[::-1], m.values[::-1], color="#264653")
for i, v in enumerate(m.values[::-1]): axes[0].text(v + 0.3, i, f"{v:.1f}", va="center", fontsize=8)
axes[0].set_xlabel("mean ms per stage (branches overlap in time)"); axes[0].set_title("Where the time goes", loc="left", fontsize=10)
axes[1].plot(t["total"].values, lw=1, color="#2a9d8f")
axes[1].axhline(1000 / 15, ls="--", c="#e76f51", lw=1); axes[1].text(2, 1000 / 15 + 2, "15 fps budget", color="#e76f51", fontsize=8)
axes[1].set_xlabel("frame"); axes[1].set_ylabel("ms"); axes[1].set_title(f"Per-frame latency: mean {lat['mean_ms']:.0f} ms = {lat['fps']:.1f} fps", loc="left", f
plt.tight_layout(); plt.savefig(FIG / "latency.png", dpi=160, bbox_inches="tight"); plt.show()
```



Notebook cell 32

10. Live webcam prototype

The same pipeline runs on the laptop webcam. The window and the optional recording only ever contain masked frames:

```
python app/run_camera.py --source 0 --profile retail --record data/runs/demo.mp4 --log data/runs/demo.csv
```

Press **g** while you are actually typing a PIN to mark ground truth; the per-frame log then gives frame-level recall, false-mask rate and FPS for your own recording. The cell below analyses the newest log if one exists.

In [19]:
```
logs = sorted((ROOT / "data" / "runs").glob("*.csv"))
if not logs:
    print("No webcam log yet. Run app/run_camera.py with --log to record one.")
else:
    log = pd.read_csv(logs[-1])
    gt, pred = log.ground_truth_pin == 1, log.pin_active == 1
    print(f"{logs[-1].name}: {len(log)} frames")
    print(f"PIN frames masked  : {(gt & pred).sum()}/{gt.sum()}  ({(gt & pred).sum() / max(gt.sum(), 1):.1%})")
    print(f"non-PIN frames masked: {(~gt & pred).sum()}/{(~gt).sum()}")
    print(f"mean latency: {log.total.mean():.1f} ms ({1000 / log.total.mean():.1f} fps)")
```

```
No webcam log yet. Run app/run_camera.py with --log to record one.
```

Notebook cell 34

11. Summary

In [20]:
```
(ROOT / "docs" / "results.json").write_text(json.dumps(RESULTS, indent=2, default=str))
lat = RESULTS["latency"]
print("Evaluation")
for k, v in RESULTS["eval"].items(): print(f"  {k:<55} {v}")
print(f"\nSpeed (640x480, laptop CPU): {lat['mean_ms']} ms mean, {lat['p95_ms']} ms p95 -> {lat['fps']} fps")
```

```
Evaluation
  Set A: sensitive regions protected (>=90% masked)       4/8
  Set A: mean coverage of sensitive regions               58.1%
  Set A: over-mask (share of frame outside truth)         3.6%
  Set B: false alarms (device, no PIN entry)              1/26
  Set C: false alarms (clothed people)                    0/21
  Sets B+C: mean share of frame masked                    0.04%

Speed (640x480, laptop CPU): 139.1 ms mean, 159.9 ms p95 -> 7.2 fps
```

Notebook cell 36

Appendix B: Data sources and licences

83 images from Wikimedia Commons. Licence counts below; per-image author, licence and page URL are in data/samples/sources.json.

Licence	Images
CC BY-SA 4.0	24
CC BY-SA 3.0	14
CC BY 2.0	12
CC0	11
CC BY 4.0	11
CC BY-SA 2.0	6
CC BY 3.0	3
OGL v1.0	1
Public domain	1

MediaPipe Pose Landmarker, Hand Landmarker and EfficientDet-Lite0 are released by Google (with model cards); and NudeNet 320n is released in NudeNet repository. Please consult each upstream licence for commercial purposes. The anatomy figure kit (wheresryan22/anatomy) is MIT licensed.